UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

CENTRO UNIVERSITARIO DEL NORTE

LENGUAJES FORMALES Y DE PROGRAMACION



PROYECTO ANALIZADOR LEXICO:

Entregable 4-5: Entrega Final.

Fase final: Autómatas AFD y AFND

APELLIDOS, NOMBRE	CARNÉ	CARRERA
Elvis Steve Xona Nolasco	202345156	ICS
Josué Artemio Lima Tiul	202442938	ICS
Kristel Paola Pop Ramírez	202444657	ICS
Frander Reginaldo Caal Xol	202151163	ICS

FECHA DE ENTREGA: 03/11/2026

INDICE

1. Introducción....3
2. Objetivos....3
3. Resumen de los entregables....3
4. Especificación léxica - Entregable 1....4
4.1 Palabras reservadas (24)....4
4.2 Tokens y expresiones regulares....4
4.3 Reglas de error....5
5. Diseño del autómata y arquitectura - Entregable 2....5
5.1 Arquitectura....6
6. Implementación del motor - Entregable 3....7
7. Interfaz gráfica - Entregable 4....7
7.1 Funcionalidades implementadas....8
7.2 Manual breve de uso....8
8. Pruebas y resultados: matriz de pruebas - Entregable 5....12
8.1 Casos del Entregable 3....12
8.2 Batería ampliada (T01 a T15)....12
8.3 Casos de las mejoras y de la fase final (T16 a T28)....13
9. Mejoras posteriores a la entrega 4-5....16
9.1 Diccionario editable y exportable....16
9.2 Errores de estructura y de operadores (reglas 8 a 11)....17
9.3 Tablas editables con efecto sobre el código....18
9.4 Exportación completa o por secciones....19
10. Fase final: autómatas finitos AFD y AFND....20
10.1 Enunciado y alcance....20
10.2 Análisis de requisitos....20
10.3 Diseño....21
10.4 Adaptabilidad según AFD o AFND....22
10.5 Manual de uso del módulo....23
10.6 Ejemplos de análisis....25
10.7 Cumplimiento de los criterios de calificación....26
11. Manual técnico....26
11.1 Requisitos y compilación....26
11.2 Flujo de datos....26
11.3 Archivos entregados....27
12. Manual de usuario....27
12.1 Instalación y ejecución....27
12.2 Uso paso a paso....27
13. Conclusiones....27
Trabajo futuro....28
14. Referencias bibliográficas....28
15. Repositorio....28

1. Introducción

En el trabajo final se integró el motor léxico del Entregable 3 con una aplicación de escritorio Windows Forms completa. La interfaz no contiene lógica de análisis: el formulario `FrmPrincipal` invoca a `AnalizadorLexico` y muestra exactamente lo que calcula el núcleo, cumpliendo la separación entre lógica e interfaz definida en el Entregable 2 (espacios de nombres `Core` y `Views`).

Este documento consolida los entregables del proyecto Analizador Léxico, un programa que se trabajó a lo largo del semestre y que lee código fuente de un subconjunto de C# y produce la lista de tokens, la tabla de símbolos, los errores y las estadísticas. El reconocimiento se hace carácter por carácter con un autómata finito determinístico escrito a mano, sin usar Regex como motor de análisis.

Sobre esa base se incorporaron cuatro mejoras solicitadas después de la entrega 4-5 (diccionario de palabras reservadas editable, detección de errores de estructura y de operadores, tablas editables sincronizadas con el editor y exportación selectiva) y la **fase final del proyecto**: un módulo para definir, dibujar y probar autómatas finitos deterministas (AFD) y no deterministas (AFND) a partir de su tabla de transiciones, abierto desde el menú principal del analizador.

2. Objetivos

Diseñar e implementar un analizador léxico con interfaz gráfica aplicando expresiones regulares y autómatas finitos determinísticos, y ampliarlo con una herramienta de visualización de AFD y AFND.

- Definir formalmente los tokens con expresiones regulares (Entregable 1).
- Diseñar el AFD y la arquitectura del sistema (Entregable 2).
- Implementar el motor léxico con tabla de símbolos y recuperación de errores (Entregable 3).
- Integrar una interfaz Windows Forms con exportación de resultados (Entregable 4).
- Probar, documentar y sustentar el sistema (Entregable 5).
- Permitir editar el diccionario de palabras reservadas, detectar errores de estructura y de operadores, editar las tablas de resultados con efecto sobre el código y exportar por secciones (mejoras posteriores).
- Construir un formulario para ingresar la tabla de transiciones de un AFD o AFND, dibujar el autómata y visualizar el análisis de hilos de una cadena de entrada (fase final).

3. Resumen de los entregables

Entregable	Producto	Resultado clave
1 – Especificación	Lenguaje C#, 24 palabras reservadas, tabla de tokens con expresiones regulares, 5 reglas de error, 3 archivos de prueba	Base formal del analizador
2 – Diseño	AFD general con S0, tablas de transición, diagrama de clases, boceto de interfaz y repositorio	Arquitectura Core / Views
3 – Motor léxico	Analizador por autómata, tabla de símbolos, recuperación de errores y suite de 16 pruebas	Se corrigió el AFD para rechazar 15.50.99 y se agregaron las reglas 6 y 7

Entregable	Producto	Resultado clave
4 – Interfaz	Aplicación Windows Forms con tokens, errores, símbolos, estadísticas y exportación	Interfaz sobre el mismo núcleo, sin modificarlo
5 – Cierre	Batería ampliada de 31 casos, manuales, informe final, release y sustentación	Matriz de pruebas con 15 casos nuevos
Mejoras posteriores	Diccionario editable, reglas 8 a 11 (estructura y operadores), tablas editables, exportación selectiva	Suite ampliada a 44 casos (T16 a T28)
Fase final – AFD / AFND	Formulario de tabla de transiciones, dibujo del autómata y análisis de hilos de una cadena	Módulo `Core.Automata` y `Views.FrmAutomatas`, abierto desde el menú principal

4. Especificación léxica - Entregable 1

Lenguaje fuente: subconjunto de C# (Opción A). C# distingue mayúsculas de minúsculas: `If`, `WHILE` o `Class` no son palabras reservadas sino identificadores.

4.1 Palabras reservadas (24)

if, else, while, for, do, switch, case, default, break, continue, return, int, float, double, char, bool, string, void, class, static, public, private, true, false. A partir de las mejoras posteriores esta lista inicial puede editarse desde el diccionario (sección 9.1).

4.2 Tokens y expresiones regulares

Token	Categoría	Expresión regular
TK_PR_PALABRA	Palabra reservada	if | else | while | … | true | false
TK_IDENTIFICADOR	Identificador	(letra | _)(letra | dígito | _)*
TK_NUM_ENTERO	Número entero	dígito+
TK_NUM_DECIMAL	Número decimal	dígito+ . dígito+
TK_CADENA	Literal cadena	" (carácter_cadena)* "
TK_CARACTER	Literal carácter	' carácter_char '
TK_OP_* aritméticos	Operadores	+ - * / % ++ -- (unión de literales)
TK_OP_* relacionales	Operadores	== != < > <= >= (unión de literales)
TK_OP_* lógicos	Operadores	&& || ! (unión de literales)
TK_OP_ASIG*	Asignación	= += -= *= /= (unión de literales)
Delimitadores	Delimitadores	( ) { } [ ] ; , . : (un carácter cada uno)
TK_COM_LINEA	Comentario de línea	// (Σ − {\n})*
TK_COM_BLOQUE	Comentario de bloque	/* … */ (hasta el primer */)

4.3 Reglas de error

Las reglas 1 a 7 son léxicas y las produce el autómata. Las reglas 8 a 11 son de estructura y operadores y las produce una segunda pasada sobre los tokens (`VerificadorEstructura`, sección 9.2), de modo que el motor léxico y sus pruebas no cambian.

Regla	Error	Recuperación / criterio
1	Identificador que inicia con dígito (1erValor, 9a)	Agrupa el lexema alfanumérico y continúa
2	Cadena sin comilla de cierre	Reporta hasta el fin de línea y sigue en la línea siguiente
3	Comentario de bloque sin cerrar	Reporta desde la apertura hasta el fin del archivo
4	Carácter no reconocido ($ # @ & | solos)	Aísla el carácter, avanza uno y continúa
5	Literal carácter mal formado (", 'abc')	Descarta hasta la comilla de cierre, ; o fin de línea
6	Número mal formado (15.50.99, 3.14abc)	Agrupa el lexema completo y continúa
7	Secuencia de escape inválida (\q)	Reporta la barra y el carácter; continúa
8	Delimitador ( ) [ ] { } sin su pareja	Se reporta el delimitador sobrante o el que quedó abierto
9	Falta ';' al final de una instrucción	Se reporta en el último token de la instrucción
10	Estructura incompleta: falta '(' tras if/while/for/switch, o '{' tras una clase o método	Se reporta en la palabra clave o en el nombre
11	Expresión mal formada: falta un operador (=, +, <, ==, &&, ||...) entre dos operandos, falta un operando, o '=' dentro de if/while	Se reporta en el operando o en el operador involucrado

Las reglas 6 y 7 son extensiones documentadas en el Entregable 3 sobre las cinco reglas del Entregable 1. Las reglas 8 a 11 son extensiones posteriores a la entrega 4-5.

5. Diseño del autómata y arquitectura - Entregable 2

Se eligió un único autómata general con estado inicial S0: según el primer carácter se deriva a S1 (identificadores y reservadas), S2 (números), S3 (cadenas y caracteres) o S4 (operadores, delimitadores y comentarios). Al refinarlo en el Entregable 3 se detectó que el diseño original de S2 (bucle con dígito o punto) aceptaría 15.50.99 como número; por eso S2 se separó en entero y decimal (el punto solo se consume si le sigue un dígito) y S3 en dentro y escape.

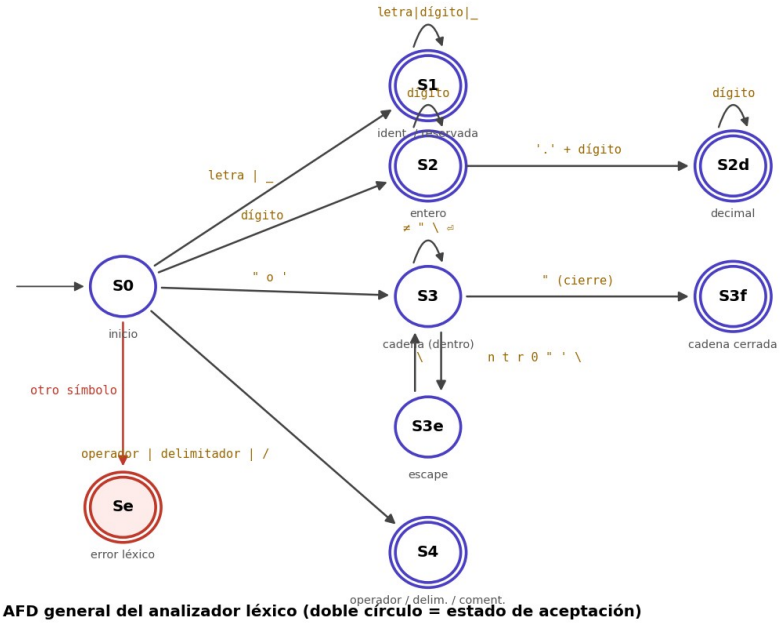


Figura 1. AFD general del analizador léxico (doble círculo = estado de aceptación).

Estado	Símbolo de entrada	Siguiente	Acción
S0	letra | _	S1	Inicia identificador o palabra reservada
S0	dígito	S2	Inicia número entero
S0	"	S3	Inicia cadena
S0	'	S3 (carácter)	Inicia literal carácter
S0	operador, delimitador o /	S4	Operador, delimitador o comentario
S0	otro símbolo	Se	Regla 4: se reporta y se continúa
S1	letra | dígito | _	S1	Extiende el lexema; al terminar consulta la tabla de reservadas
S2	dígito	S2	Extiende el entero
S2	"." seguido de dígito	S2d	Pasa a decimal
S2d	dígito	S2d	Extiende el decimal
S3	cualquiera salvo " \ salto	S3	Consume contenido
S3	\	S3e	Inicia escape
S3e	n t r 0 " ' \	S3	Escape válido (otro: Regla 7)
S3	"	S3f	Cierra la cadena y emite TK_CADENA
S3	salto o EOF	Se	Regla 2: cadena sin cerrar

5.1 Arquitectura

Capa / clase	Responsabilidad
Core.AnalizadorLexico	Autómata S0 a S4; lectura carácter por carácter con línea y columna

Capa / clase	Responsabilidad
Core.Token, TiposToken	Tipo, lexema, línea, columna y categoría
Core.PalabrasReservadas	Diccionario de palabras reservadas y tipos de dato (comparación ordinal); editable en ejecución
Core.TablaSimbolos, ElementoSimbolo	Diccionario por lexema sin duplicados; frecuencia y referencias
Core.ManejadorErrores, ErrorLexico	Errores con regla, posición y descripción; el análisis nunca se detiene
Core.VerificadorEstructura	Segunda pasada: reglas 8 a 11 (delimitadores, ";", "(" "{" y operadores)
Core.ResultadoAnalisis, Reporte	Estadísticas y reportes de texto
Core.Exportador	CSV, HTML y escritura en UTF-8
Core.Automata, Hilo, PasoHilo, ResultadoCadena	Modelo de AFD/AFND: tabla de transiciones, validación, cierre-ε, análisis de cadenas y enumeración de hilos (fase final)
Views.FrmPrincipal	Diseño en FrmPrincipal.Designer.cs; lógica de eventos, despliegue y edición de resultados en FrmPrincipal.cs
Views.ResaltadorSintaxis	Colorea el editor con los tokens y errores del motor
Views.FrmDiccionario	Ventana del diccionario de palabras reservadas (editar, importar, exportar)
Views.DlgExportar	Diálogo para elegir qué exportar: todo o una sección
Views.FrmAutomatas, PanelAutomata, DatosProyecto	Formulario de tabla de transiciones AFD/AFND, control que dibuja el autómata y datos del proyecto (fase final)
Views.Nativos	Llamadas a Windows para sincronizar el desplazamiento y evitar parpadeo

6. Implementación del motor - Entregable 3

- Un único autómata con S0 central; cada estado es un método con switch/case.
- Mayor coincidencia con un carácter de anticipación (+=, ++, ==, <=, &&, //, /*).
- La tabla de símbolos solo guarda identificadores; el tipo de dato se infiere únicamente si sigue a un tipo primitivo del diccionario.
- Recuperación de errores: cada regla aísla el lexema inválido, lo reporta con línea y columna y continúa.
- Línea y columna comienzan en 1; \n reinicia la columna, \r no cuenta y el tabulador cuenta como una columna.

7. Interfaz gráfica - Entregable 4

La interfaz se diseñó con el diseñador de Windows Forms e integra el núcleo sin modificarlo: editor con numeración de líneas y resaltado, tabla de tokens, panel de errores en rojo, tabla de símbolos, estadísticas y exportación. Los errores se muestran con línea y columna y un clic lleva al lugar exacto en el editor. La barra superior incluye ahora los botones Diccionario y Autómatas AFD / AFND.

7.1 Funcionalidades implementadas

Requisito del enunciado	Implementación
Control de texto para el código	RichTextBox con numeración de líneas al costado y resaltado de sintaxis opcional
Cargar archivo (OpenFileDialog)	Botón Abrir archivo (Ctrl+O) y arrastrar un archivo sobre el editor
Botón Analizar	Botón Analizar (F5); mensaje claro si no hay código
DataGridView: #, Lexema, Tipo, Línea, Columna	Pestaña Tokens con columnas #, Lexema, Tipo de token, Categoría, Línea y Columna; un clic en una fila selecciona el lexema en el editor
Panel de errores distinguible	Pestaña Errores con filas en rojo y contador en el nombre de la pestaña; incluye las reglas 1 a 11
Tabla de símbolos	Pestaña Tabla de símbolos: Id, lexema, tipo de token, tipo de dato, primera línea/columna, frecuencia y referencias
Panel de estadísticas	Pestaña Estadísticas: total de tokens por categoría, líneas, errores, identificadores distintos y tiempo; resumen también en la barra de estado
Exportar (SaveFileDialog)	Diálogo para elegir todo (.txt o .html) o solo tokens, errores o símbolos (.csv o .txt) o estadísticas (.txt)
Usabilidad y excepciones	Mensajes para código vacío, archivo binario, grande (más de 5 MB), sin permisos o en uso; manejador global de excepciones
Opcional: resaltado de sintaxis	Implementado con los mismos tokens del motor (sin Regex): palabras reservadas en azul y negrita, cadenas en rojo oscuro, comentarios en verde y errores con fondo rosado
Mejora: diccionario	Botón Diccionario: ventana para editar, importar y exportar las palabras reservadas
Mejora: tablas editables	Tokens, errores y símbolos se editan en la propia tabla y el cambio se aplica al código (sección 9.3)
Fase final: AFD / AFND	Botón Autómatas AFD / AFND: abre el formulario de tabla de transiciones y dibujo (sección 10)

La ventana se diseñó en el diseñador de Visual Studio (FrmPrincipal.cs [Diseño]); los formularios FrmDiccionario, DlgExportar y FrmAutomatas también tienen su archivo .Designer.cs y se pueden modificar visualmente. El código de los formularios solo contiene la lógica de los eventos.

7.2 Manual breve de uso

7.2.1 Pantalla principal

A la izquierda está el editor y a la derecha las pestañas de resultados. La barra superior tiene los botones Abrir archivo, Analizar, Exportar, Limpiar, Diccionario y Autómatas AFD / AFND. La barra inferior muestra mensajes, la posición del cursor y un resumen del último análisis.

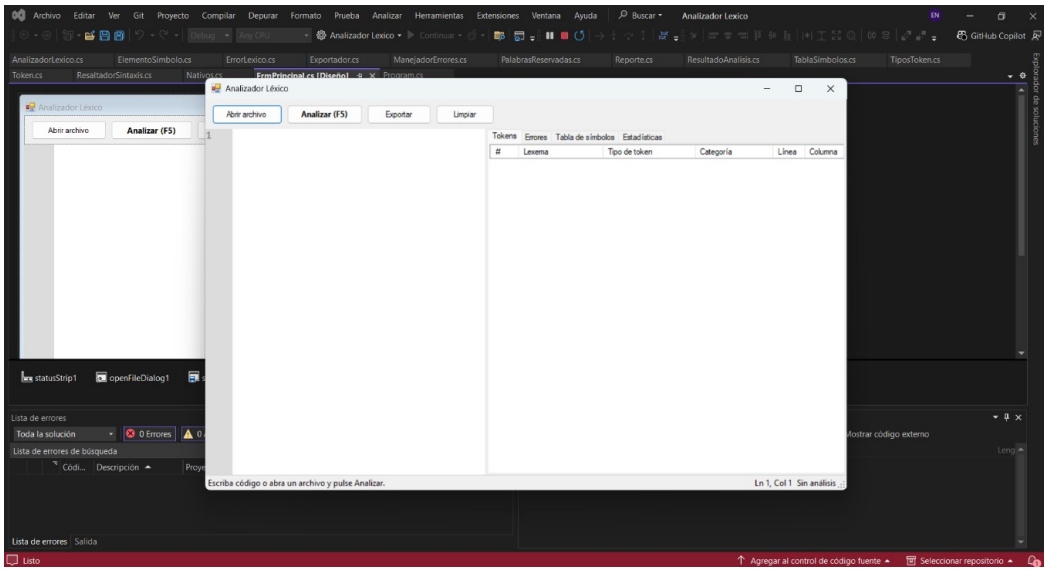


Figura 2. Ventana principal del Entregable 4 (antes de agregar los botones Diccionario y Autómatas).

7.2.2 Cargar código

1. Pulse Abrir (Ctrl+O), seleccione un archivo .txt o .cs, o arrástrelo al editor.
2. También puede escribir el código directamente en el editor.

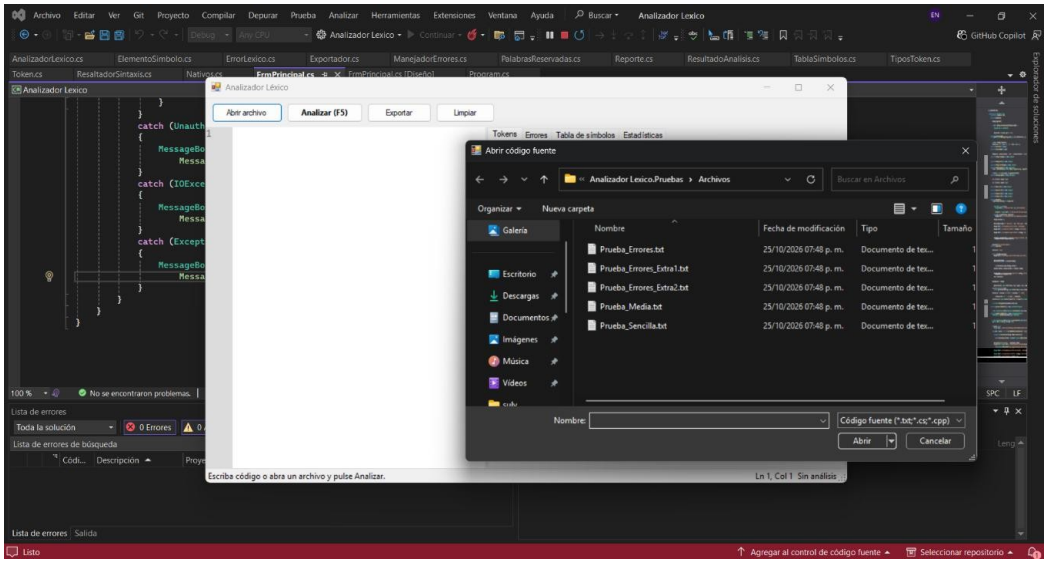


Figura 3. Diálogo para abrir un archivo de código.

7.2.3 Analizar y revisar tokens

1. Pulse Analizar (F5).
2. Un clic en una fila selecciona el lexema en el editor.

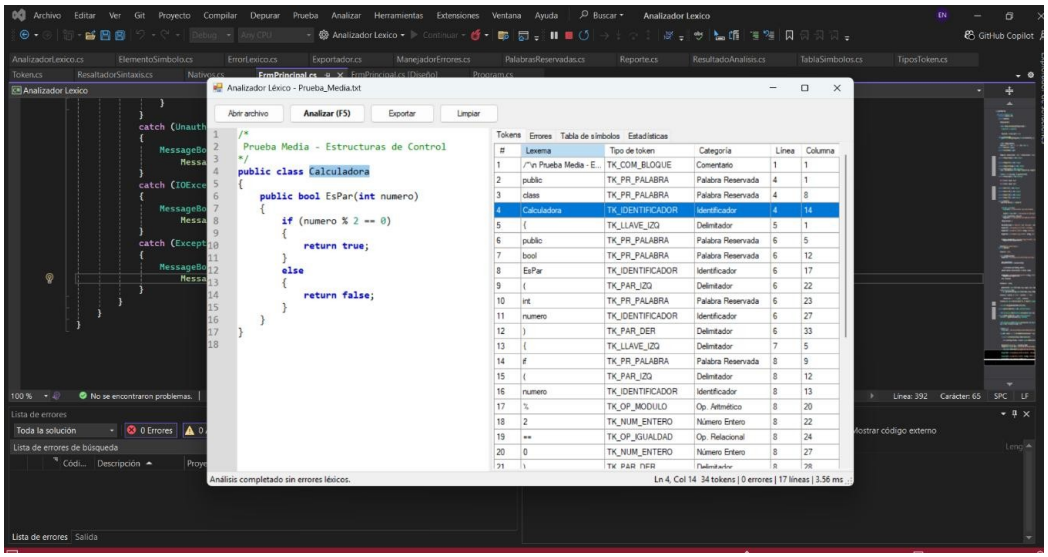


Figura 4. Tabla de tokens después de analizar Prueba_Media.txt.

7.2.4 Errores

Si hay errores, la aplicación abre la pestaña Errores, que muestra regla, lexema, línea, columna y descripción. En el editor el texto erróneo se marca en rojo.

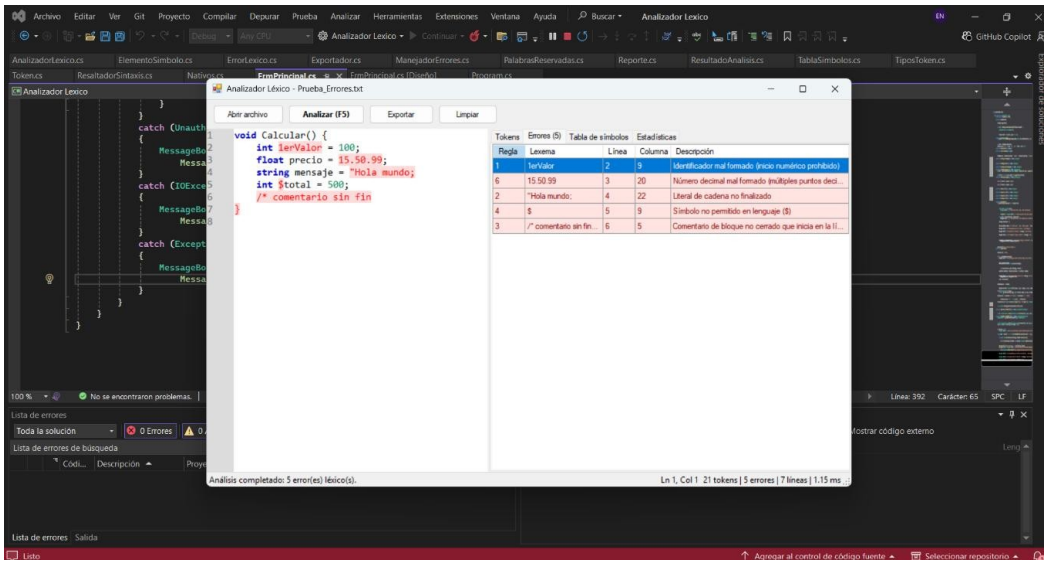


Figura 5. Errores léxicos de Prueba_Errores.txt.

7.2.5 Tabla de símbolos y estadísticas

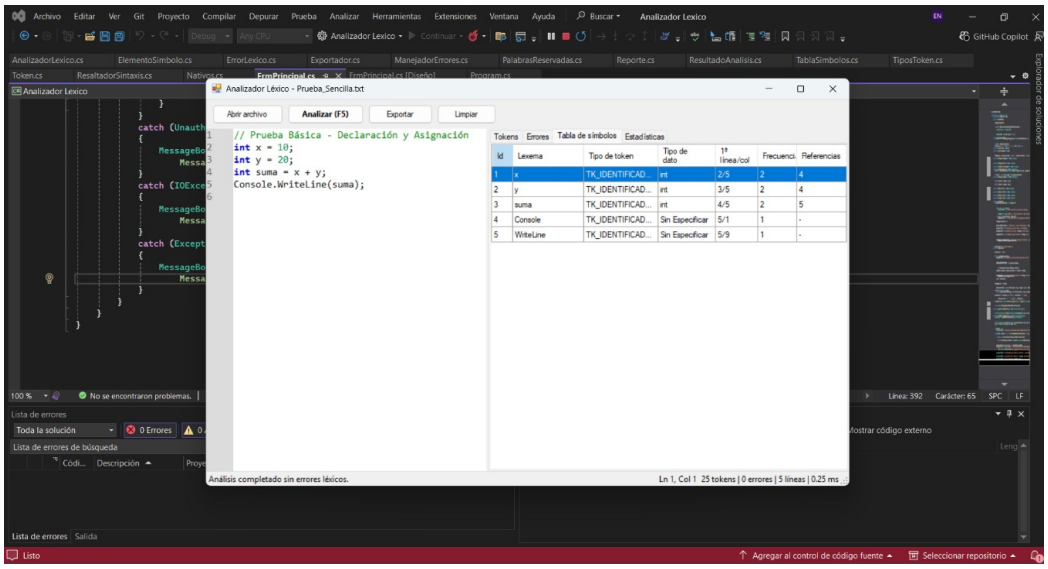


Figura 6. Tabla de símbolos de Prueba_Sencilla.txt.

7.2.6 Exportar

Pulse Exportar (Ctrl+E), elija qué exportar (sección 9.4) y el formato en el cuadro de diálogo. La aplicación ofrece abrir el archivo generado.

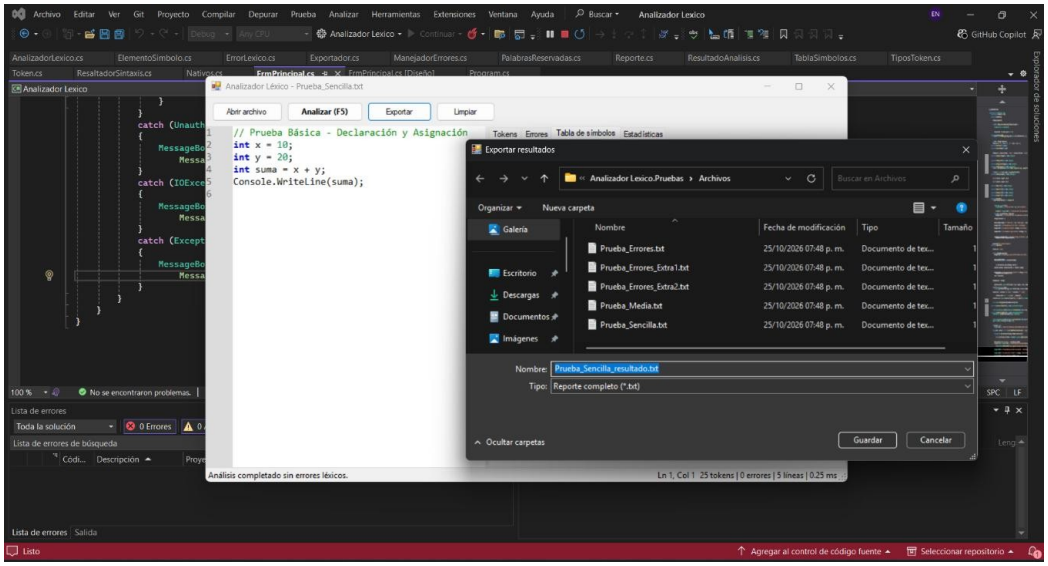


Figura 7. Cuadro de diálogo de exportación (versión del Entregable 4).

7.2.7 Atajos

Atajo	Acción
F5	Analizar
Ctrl+O	Abrir archivo
Ctrl+E	Exportar resultados

8. Pruebas y resultados: matriz de pruebas - Entregable 5

La suite es un programa de consola (proyecto Analizador Lexico.Pruebas) con **44 casos**: los 16 del Entregable 3, 15 del Entregable 5 (T01 a T15) y 13 de las mejoras y la fase final (T16 a T28). Cada caso valida conteos y posiciones exactas y los resultados se guardan en resultados_pruebas.txt. En la última ejecución los 44 casos resultaron aprobados.

8.1 Casos del Entregable 3

Caso	Entrada	Esperado	Obtenido
P01	Prueba_Sencilla.txt	25 tokens, 0 errores, 5 símbolos	25 / 0 / 5
P02	Prueba_Media.txt	34 tokens, 0 errores, 3 símbolos	34 / 0 / 3
P03	Prueba_Errores.txt	21 tokens, 5 errores, 4 símbolos	21 / 5 / 4
P04	Errores extra 1	28 tokens, 4 errores, 8 símbolos	28 / 4 / 8
P05	Errores extra 2	24 tokens, 5 errores, 3 símbolos	24 / 5 / 3
P06 a P16	Mayúsculas, operadores, números, escapes, CRLF, vacío, "/*/", símbolos, comentario final, reanálisis, no ASCII	Según Entregable 3	Todos aprobados (16 de 16)

8.2 Batería ampliada (T01 a T15)

Caso	Entrada	Salida esperada	Salida obtenida	Resultado
T01	Las 24 palabras reservadas separadas por espacio	24 tokens TK_PR_PALABRA, 0 errores, 0 símbolos	24 tokens, 0 errores, 0 símbolos	Aprobado
T02	Identificador de 500 letras, "_" y "a1_b2"	3 identificadores, 3 símbolos, 0 errores; "_" en (1,502)	3 tokens, 0 errores, 3 símbolos	Aprobado
T03	3.14159265358979323846 0 007 12.0	Decimal, entero, entero, decimal; 0 errores	4 tokens, 0 errores, 0 símbolos	Aprobado
T04	"" "a // b" "tab\there" "q\"uote"	4 cadenas, ningún comentario, 0 errores	4 tokens, 0 errores, 0 símbolos	Aprobado
T05	'a' '9' '\n' '\\' '\''	5 caracteres literales, 0 errores	5 tokens, 0 errores, 0 símbolos	Aprobado
T06	a+++b x=-1 i--- j	Lexemas: a ++ + b x = - 1 i -- - j; 0 errores	12 tokens, 0 errores, 5 símbolos	Aprobado
T07	// /* no abre · /* a /* b */ c */ int	6 tokens: comentario línea, bloque (termina en 1.er */), c, *, /, int	6 tokens, 0 errores, 1 símbolos	Aprobado
T08	int a; + comentario /* sin cerrar (línea 2)	3 tokens, 1 error Regla 3 en (2,1)	3 tokens, 1 errores, 1 símbolos [R3(2,1)]	Aprobado
T09	x = 9a + 3.1.2 # 'xy' @ "sin	3 tokens válidos y 6 errores: R1, R6, R4, R5, R4, R2	3 tokens, 6 errores, 1 símbolos [R1(1,5),R6(1,10),R4(1,16),R5(1,18),R4(1,23),R2(1,25)]	Aprobado
T10	( ) { } [ ] ; , . :	10 delimitadores; ":" en (1,19)	10 tokens, 0 errores, 0 símbolos	Aprobado
T11	s = "abc🐍 def";	3 tokens, 2 errores R2 en (1,5) y (2,5)	3 tokens, 2 errores, 2 símbolos [R2(1,5),R2(2,5)]	Aprobado

Caso	Entrada	Salida esperada	Salida obtenida	Resultado
T12	10 000 líneas "int vN = N;"	50 000 tokens, 10 000 símbolos, 0 errores, < 5 s	50000 tokens, 0 errores, 10000 símbolos	Aprobado
T13	Exportación CSV de a = "," ;	5 filas; "=" protegido con apóstrofo; cadena con comillas dobladas	Coincide con lo esperado (verificado por la prueba automática)	Aprobado
T14	Exportación HTML con "<", "&&" y título <b>	Entidades escapadas; sin etiquetas inyectadas	Coincide con lo esperado (verificado por la prueba automática)	Aprobado
T15	BOM + tabulador + int x;	3 tokens, 0 errores; int en (1,2)	3 tokens, 0 errores, 1 símbolos	Aprobado

8.3 Casos de las mejoras y de la fase final (T16 a T28)

Caso	Entrada	Salida esperada	Salida obtenida	Resultado
T16	int x = 5␏int y = 6;	1 error Regla 9 en (1,9); la línea 2 es válida	1 error [R9(1,9)]	Aprobado
T17	void f() {␏ int x = 1; / int x = 1; }	'{' sin cerrar en (1,10); '}' sin abrir en (1,12)	[R8(1,10)] y [R8(1,12)]	Aprobado
T18	if x > 3 { } / class A␏int x;	Falta '(' en (1,1); falta '{' de la clase en (1,7)	[R10(1,1)] y [R10(1,7)]	Aprobado
T19	int x 5; · x = ; · y = a +; · if (x = 5) · if (a b)	R11 en (1,5), (1,3), (1,7), (1,7) y (1,5)	Cinco errores R11 en las posiciones esperadas	Aprobado
T20	Prueba_Sencilla.txt y Prueba_Media.txt	0 errores de estructura (sin falsos positivos)	0 errores en ambos	Aprobado
T21	Diccionario {entero*, si} y "entero x = 5; int y;"	entero reservada y tipo de dato; int pasa a identificador; x:entero; al restaurar vuelven 24	Coincide	Aprobado
T22	AFD "termina en ab": aab, abb, vacía, abc	aab acepta; abb y vacía rechazan; abc error de alfabeto; 1 hilo	Coincide; hilo q0 -a-> q1 -a-> q1 -b-> q2	Aprobado
T23	AFD parcial (celda vacía) con "ab"	Rechaza; el hilo queda Bloqueado	Rechaza; hilo Bloqueado	Aprobado
T24	AFND "termina en ab" con "ab" y "ba"	ab: acepta con 2 hilos (1 acepta); conjuntos {q0,q1} y {q0,q2}; ba rechaza	Coincide	Aprobado
T25	AFND con ε (Σ*ab) con "aab"	Cierre-ε de q0 = {q0,q1}; acepta; algún hilo usa ε y termina en q3	Coincide	Aprobado
T26	AFD con 2 destinos, sin inicial, estado inexistente	La validación reporta los tres problemas	Coincide	Aprobado
T27	q0 -a-> q0, q0 -b-> q0,q1	2 aristas; el bucle se etiqueta "a,b"	Coincide	Aprobado
T28	AFND de bifurcación total con a^12	Acepta; lista truncada en 300 hilos	Acepta; truncado; 300 hilos	Aprobado

Cómo se obtuvieron los valores. Las salidas esperadas se derivaron de las expresiones regulares y reglas del Entregable 1 y del comportamiento definido para las reglas 8 a 11 y para los autómatas,

no del programa. Las salidas obtenidas provienen de ejecutar la suite completa de 44 casos sobre el código final (resultado: 44 aprobados, 0 fallidos). Antes de entregar, ejecute la suite en Visual Studio con Ctrl+F5 en Analizador Lexico.Pruebas:

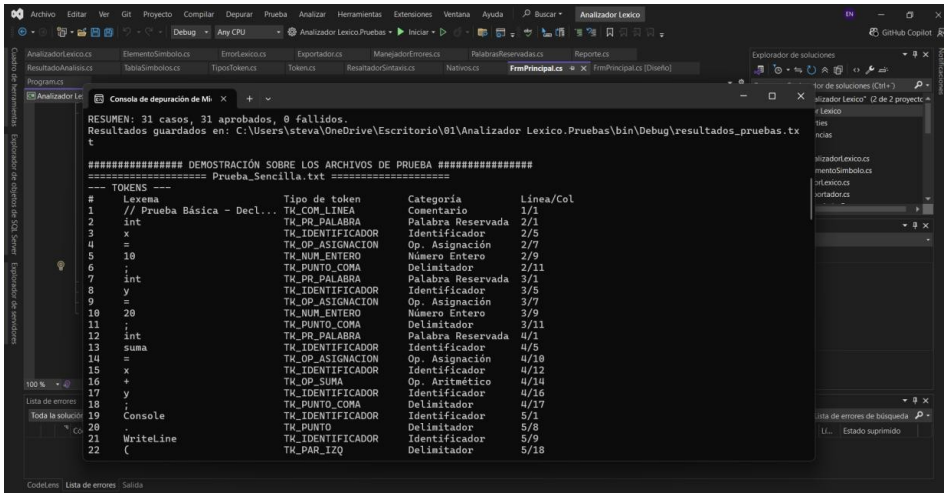


Figura 8. Resumen de la suite y demostración sobre Prueba_Sencilla.txt.

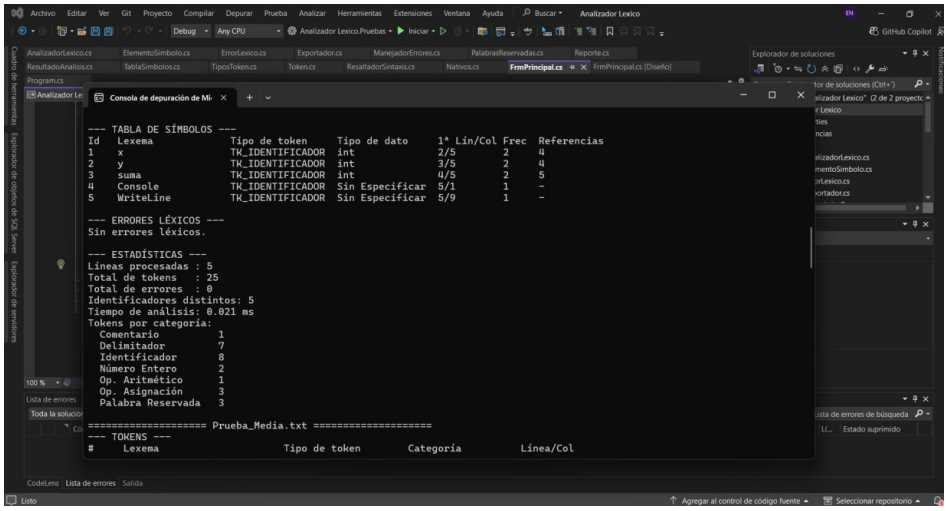
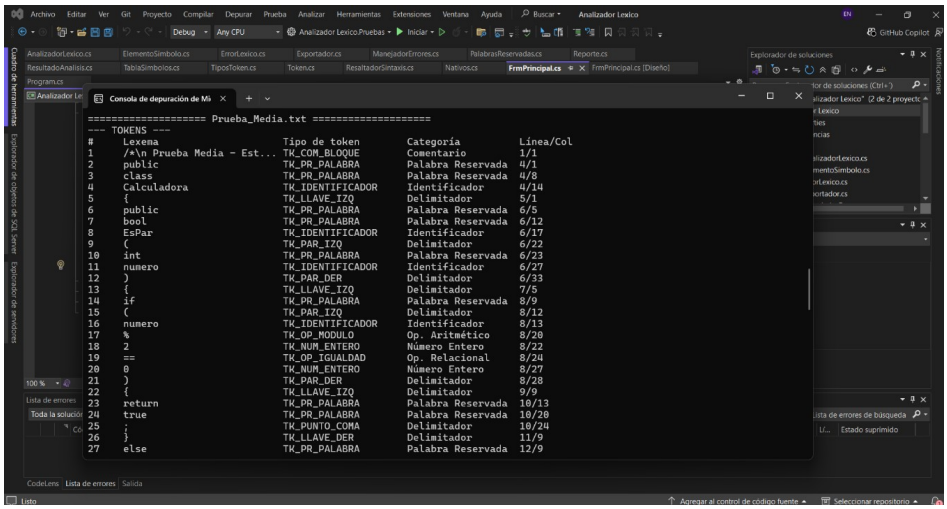
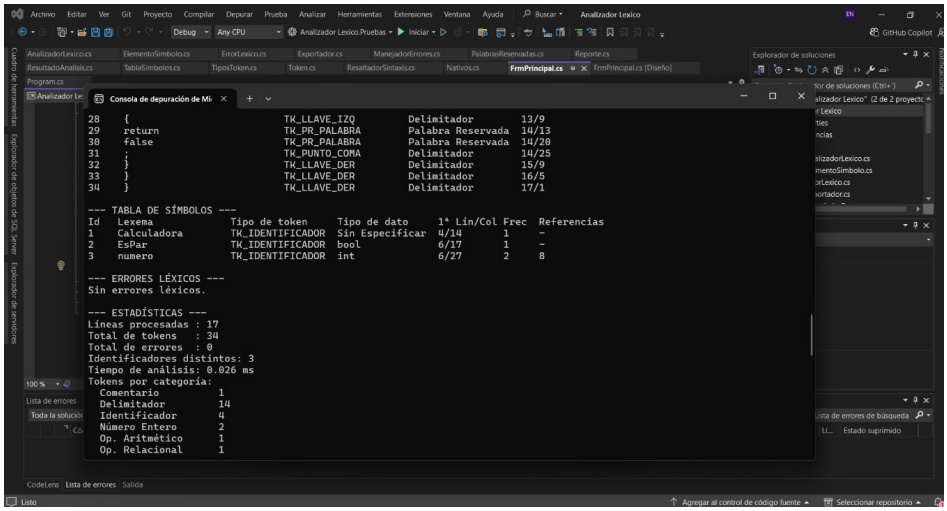


Figura 9. Tabla de símbolos, errores y estadísticas de Prueba_Sencilla.txt.



```
================== Prueba_Media.txt ==================
--- TOKENS ---
#    Lexema                  Tipo de token        Categoría          Línea/Col
1    /*\n Prueba Media - Est... TK_COM_BLOQUE       Comentario         1/1
2    public                  TK_PR_PALABRA        Palabra Reservada  4/1
3    class                   TK_PR_PALABRA        Palabra Reservada  4/8
4    Calculadora             TK_IDENTIFICADOR     Identificador      4/14
5    {                       TK_LLAVE_IZQ         Delimitador        5/1
6    public                  TK_PR_PALABRA        Palabra Reservada  6/5
7    bool                    TK_PR_PALABRA        Palabra Reservada  6/12
8    EsPar                   TK_IDENTIFICADOR     Identificador      6/17
9    (                       TK_PAR_IZQ           Delimitador        6/22
10   int                     TK_PR_PALABRA        Palabra Reservada  6/23
11   numero                  TK_IDENTIFICADOR     Identificador      6/27
12   )                       TK_PAR_DER           Delimitador        6/33
13   {                       TK_LLAVE_IZQ         Delimitador        7/5
14   if                      TK_PR_PALABRA        Palabra Reservada  8/9
15   (                       TK_PAR_IZQ           Delimitador        8/12
16   numero                  TK_IDENTIFICADOR     Identificador      8/13
17   %                       TK_OP_MODULO         Op. Aritmético     8/20
18   2                       TK_NUM_ENTERO        Número Entero      8/22
19   ==                      TK_OP_IGUALDAD       Op. Relacional     8/24
20   0                       TK_NUM_ENTERO        Número Entero      8/27
21   )                       TK_PAR_DER           Delimitador        8/28
22   {                       TK_LLAVE_IZQ         Delimitador        9/9
23   return                  TK_PR_PALABRA        Palabra Reservada  10/13
24   true                    TK_PR_PALABRA        Palabra Reservada  10/20
25   ;                       TK_PUNTO_COMA        Delimitador        10/24
26   }                       TK_LLAVE_DER         Delimitador        11/9
27   else                    TK_PR_PALABRA        Palabra Reservada  12/9
```

Figura 10. Tokens de Prueba_Media.txt.



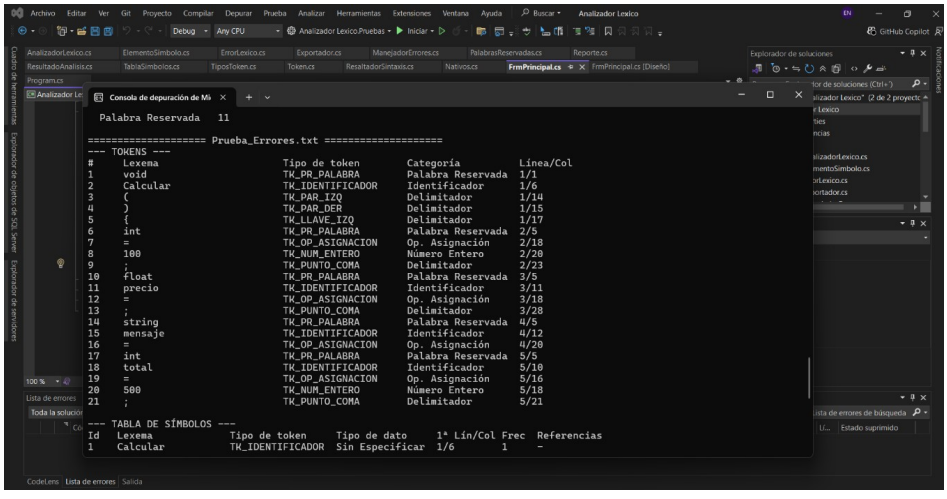
```
28   {                       TK_LLAVE_IZQ         Delimitador        13/9
29   return                  TK_PR_PALABRA        Palabra Reservada  14/13
30   false                   TK_PR_PALABRA        Palabra Reservada  14/20
31   ;                       TK_PUNTO_COMA        Delimitador        14/25
32   }                       TK_LLAVE_DER         Delimitador        15/9
33   }                       TK_LLAVE_DER         Delimitador        16/5
34   }                       TK_LLAVE_DER         Delimitador        17/1

--- TABLA DE SÍMBOLOS ---
Id   Lexema          Tipo de token     Tipo de dato     1ª Lín/Col Frec  Referencias
1    Calculadora     TK_IDENTIFICADOR  Sin Especificar  4/14       1     -
2    EsPar           TK_IDENTIFICADOR  bool             6/17       1     -
3    numero          TK_IDENTIFICADOR  int              6/27       2     8

--- ERRORES LÉXICOS ---
Sin errores léxicos.

--- ESTADÍSTICAS ---
Líneas procesadas : 17
Total de tokens   : 34
Total de errores  : 0
Identificadores distintos: 3
Tiempo de análisis: 0.026 ms
Tokens por categoría:
  Comentario        1
  Delimitador       14
  Identificador     4
  Número Entero     2
  Op. Aritmético    1
  Op. Relacional    1
```

Figura 11. Tabla de símbolos y estadísticas de Prueba_Media.txt.



```
  Palabra Reservada   11

================== Prueba_Errores.txt ==================
--- TOKENS ---
#    Lexema                  Tipo de token        Categoría          Línea/Col
1    void                    TK_PR_PALABRA        Palabra Reservada  1/1
2    Calcular                TK_IDENTIFICADOR     Identificador      1/6
3    (                       TK_PAR_IZQ           Delimitador        1/14
4    )                       TK_PAR_DER           Delimitador        1/15
5    {                       TK_LLAVE_IZQ         Delimitador        1/17
6    int                     TK_PR_PALABRA        Palabra Reservada  2/5
7    =                       TK_OP_ASIGNACION     Op. Asignación     2/18
8    100                     TK_NUM_ENTERO        Número Entero      2/20
9    ;                       TK_PUNTO_COMA        Delimitador        2/23
10   float                   TK_PR_PALABRA        Palabra Reservada  3/5
11   precio                  TK_IDENTIFICADOR     Identificador      3/11
12   =                       TK_OP_ASIGNACION     Op. Asignación     3/18
13   ;                       TK_PUNTO_COMA        Delimitador        3/28
14   string                  TK_PR_PALABRA        Palabra Reservada  4/5
15   mensaje                 TK_IDENTIFICADOR     Identificador      4/12
16   =                       TK_OP_ASIGNACION     Op. Asignación     4/20
17   int                     TK_PR_PALABRA        Palabra Reservada  5/5
18   total                   TK_IDENTIFICADOR     Identificador      5/10
19   =                       TK_OP_ASIGNACION     Op. Asignación     5/16
20   500                     TK_NUM_ENTERO        Número Entero      5/18
21   ;                       TK_PUNTO_COMA        Delimitador        5/21

--- TABLA DE SÍMBOLOS ---
Id   Lexema          Tipo de token     Tipo de dato     1ª Lín/Col Frec  Referencias
1    Calcular        TK_IDENTIFICADOR  Sin Especificar  1/6        1     -
```

Figura 12. Tokens de Prueba_Errores.txt.

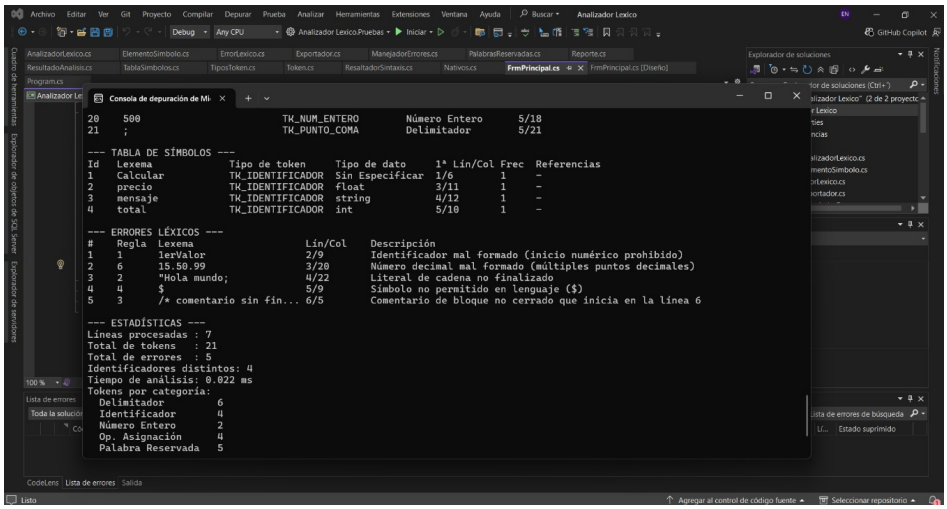


Figura 13. Errores léxicos y estadísticas de Prueba_Errores.txt.

9. Mejoras posteriores a la entrega 4-5

Después de la entrega 4-5 se solicitaron cuatro cambios. Todos se implementaron sin modificar el autómata léxico, de modo que los 31 casos anteriores siguen aprobados.

9.1 Diccionario editable y exportable

Qué es el diccionario. Es la tabla de palabras reservadas del lenguaje. Cuando el autómata termina de leer una palabra (estado S1) la busca en el diccionario: si está, emite TK_PR_PALABRA; si no, es un identificador y entra a la tabla de símbolos. Cada palabra tiene además la marca «es tipo de dato», que usa la tabla de símbolos para inferir el tipo de una variable (`int x` guarda `x:int`).

Por qué se necesita. Lo que distingue a un lenguaje de otro es su vocabulario. Antes la lista estaba fija en el código; ahora es una tabla en memoria (`Core.PalabrasReservadas`) que el usuario puede cambiar sin recompilar. Esto permite probar el analizador con otro subconjunto del lenguaje o con pseudocódigo en español, y cambia lo que se ve en Tokens, resaltado, Estadísticas y tabla de símbolos. El diccionario solo afecta a las palabras: los operadores y delimitadores siguen fijos en el autómata.

Uso. El botón Diccionario abre una ventana con las palabras. Se puede escribir en la última fila para agregar, eliminar filas, restaurar las 24 originales, importar desde .csv o .txt, exportar a .csv o .txt y aplicar. Al aplicar se vuelve a analizar el código abierto. Se validan los nombres (letras, dígitos y guion bajo, sin iniciar con dígito) y los duplicados.

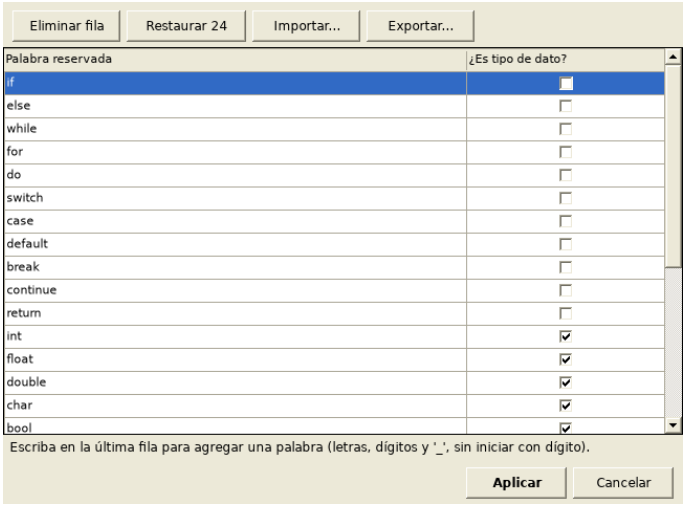


Figura 14. Ventana del diccionario con las 24 palabras iniciales.

Formatos de archivo. En .csv: encabezado `Palabra,EsTipoDeDato` y una fila por palabra (1 = tipo de dato). En .txt: una palabra por línea y, si es tipo de dato, un tabulador y `*`. Se entregan cuatro ejemplos listos para importar:

Archivo	Palabras	Descripción
diccionario_basico_24.txt	24	Las palabras originales del Entregable 1, con los 6 tipos de dato marcados
diccionario_ampliado.txt	39	Las 24 más foreach, new, null, namespace, using, try, catch, finally, throw, long, short, byte, decimal, object y var (los últimos 5 como tipos)
diccionario_minimo.csv	10	Solo if, else, while, for, return, int, float, string, true y false
diccionario_pseudocodigo_es.csv	20	Pseudocódigo en español: si, sino, mientras, para, entero, decimal, cadena, verdadero, falso...

Ejemplo real: con el diccionario de pseudocódigo, en `entero x = 5;` la palabra `entero` pasa a ser reservada, `x` queda con tipo de dato `entero` en la tabla de símbolos e `int` deja de ser reservada y se trata como identificador.

9.2 Errores de estructura y de operadores (reglas 8 a 11)

La clase `Core.VerificadorEstructura` recorre los tokens ya reconocidos (sin comentarios) y agrega sus hallazgos a la lista de errores, que se ordena por línea y columna. Es una verificación heurística para el subconjunto de C# del proyecto, no un analizador sintáctico completo.

Regla	Qué detecta	Cómo lo detecta
8	Delimitadores sin pareja	Una pila de delimitadores abiertos. Un cierre sin apertura se reporta en el cierre; un delimitador que queda abierto (al cerrar otro o al terminar el archivo) se reporta en su apertura, con la línea donde se abrió.
9	Falta ';'	Cuando el último token de una instrucción (identificador, literal, ')', ']', ++, --, break, continue) va seguido de una nueva instrucción en otra línea, de '}' o de una palabra clave en la misma línea. No exige ';' dentro de paréntesis, tras la cabecera de if/while/for/switch, ni dentro de inicializadores.

Regla	Qué detecta	Cómo lo detecta
10	Falta '(' o '{'	if, while, for, switch y foreach deben ir seguidos de '('. Una clase o un método declarado deben ir seguidos de '{'.
11	Operadores y operandos	Dos operandos juntos (falta =, +, <, ==, &&, ||...); un operador sin operando a la izquierda o a la derecha; y '=' dentro de la condición de if o while (probablemente era '=='). El mensaje sugiere los operadores posibles según el contexto: en una condición propone ==, !=, <, >, <=, >=, && y ||; en una declaración propone '='.

Para evitar errores en cascada, no se reporta ';' ni operador faltante cuando un error léxico ya eliminó un lexema entre los dos tokens, y no se reportan delimitadores abiertos al final del archivo si un comentario de bloque sin cerrar se tragó el resto del código. Los símbolos | y & sueltos siguen siendo errores léxicos de la Regla 4.

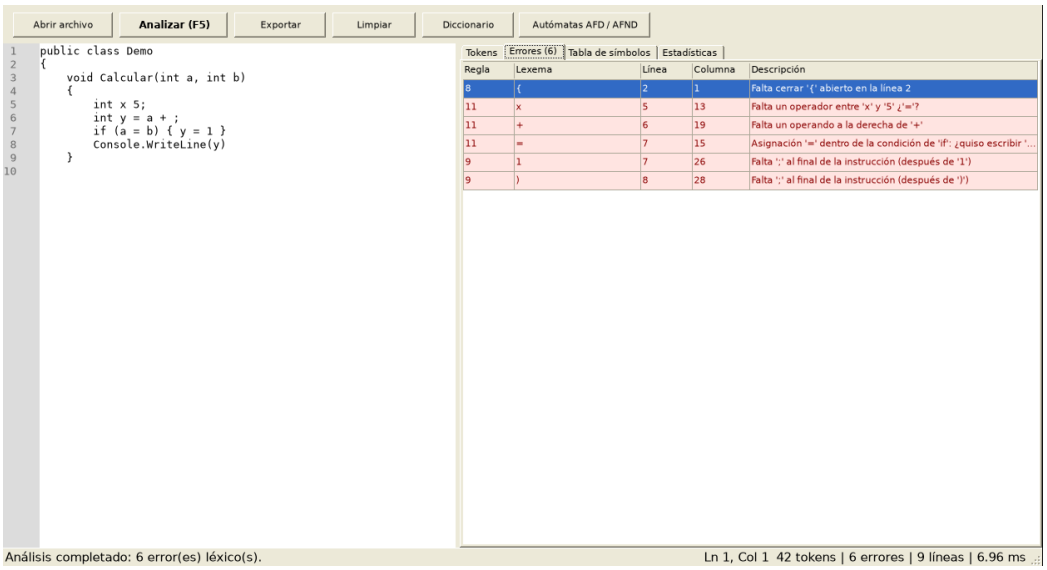


Figura 15. Errores de las reglas 8, 9 y 11 detectados sobre un código de ejemplo, y botones nuevos de la barra superior.

9.3 Tablas editables con efecto sobre el código

Las tablas Tokens, Errores y Símbolos son editables y el cambio se aplica al texto del editor; luego se vuelve a analizar para que todas las tablas queden coherentes. Así, lo que muestran las tablas siempre es consistente con el código.

Tabla	Qué se puede editar y qué ocurre
dgvTokens	La columna Lexema: se reemplaza el token en su línea y columna dentro del editor. Tipo, categoría, línea y columna se recalculan al volver a analizar.
dgvErrores	La columna Lexema: se reemplaza el texto erróneo (por ejemplo, cambiar 5 por 5; corrige una Regla 9) y el error desaparece si quedó resuelto.
dgvSimbolos	Lexema: renombra el identificador en todas sus apariciones (se valida que sea un identificador válido y no una palabra reservada). Tipo de dato: cambia el tipo en la declaración del identificador.

Tabla	Qué se puede editar y qué ocurre
txtEstadisticas	Es editable y el texto editado se conserva y se exporta. Como las estadísticas son valores calculados a partir del código, no existe una correspondencia que permita reflejarlas en el editor.

Un clic en una fila selecciona el texto en el editor; el doble clic en una celda editable abre la edición.

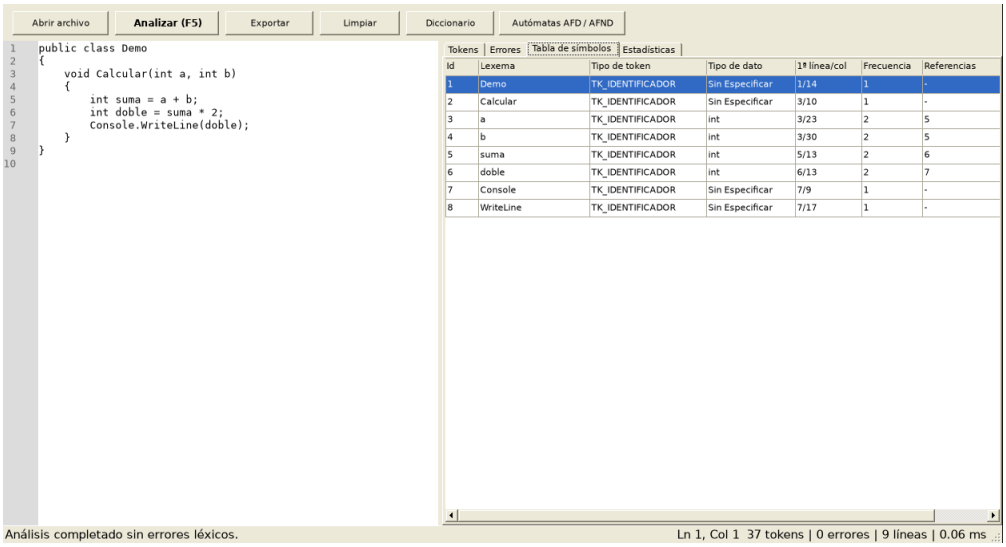


Figura 16. Antes de editar: la variable "suma" aparece en las líneas 5 y 6.

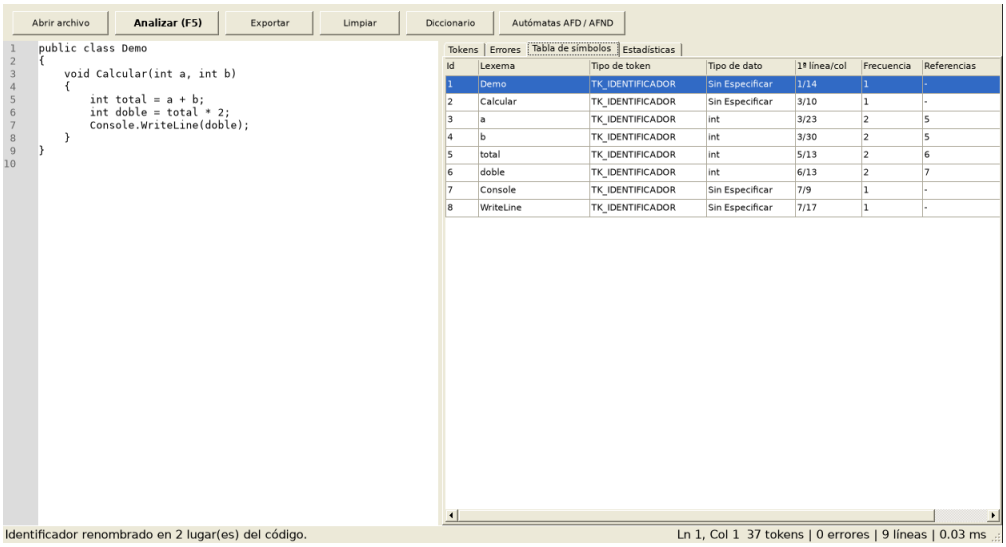


Figura 17. Después de cambiar el lexema "suma" por "total" en dgvSimbolos: el editor se actualizó en ambos lugares.

9.4 Exportación completa o por secciones

Al pulsar Exportar (Ctrl+E) aparece un diálogo para elegir qué exportar y después el cuadro para guardar, con los formatos que corresponden a la opción elegida.

Opción	Formatos	Contenido
Todo (reporte completo)	.txt, .html	Tokens, errores, tabla de símbolos y estadísticas

Opción	Formatos	Contenido
Solo tokens (dgvTokens)	.csv, .txt	Tabla de tokens
Solo errores (dgvErrores)	.csv, .txt	Tabla de errores (reglas 1 a 11)
Solo símbolos (dgvSimbolos)	.csv, .txt	Tabla de símbolos
Solo estadísticas (txtEstadisticas)	.txt	Texto de las estadísticas, incluyendo ediciones



Figura 18. Diálogo para elegir qué exportar.

10. Fase final: autómatas finitos AFD y AFND

10.1 Enunciado y alcance

El enunciado de la fase final pide una herramienta para ingresar la tabla de transiciones de un autómata finito determinista (AFD) o no determinista (AFND), dibujar sus estados y transiciones, identificar con claridad el estado inicial y los finales, y visualizar el análisis de hilos de una cadena de entrada. Debe llamarse desde el menú principal del analizador léxico y el formulario debe adaptarse a la elección del usuario.

10.2 Análisis de requisitos

N.º	Requisito del enunciado	Cómo se cumple
R1	Se llama desde el menú principal	Botón «Autómatas AFD / AFND» en la barra superior de FrmPrincipal
R2	Elegir AFD o AFND	Botones de opción AFD y AFND; el formulario se adapta (sección 10.4)
R3	Definir n estados y m símbolos de entrada según el alfabeto	Campos n (1 a 15) y m (1 a 10) y cuadro de alfabeto editable; la tabla se reconstruye al cambiarlos
R4	Llenar la tabla de transiciones	DataGridView con un renglón por estado y una columna por símbolo (y ε en AFND)
R5	Indicar estado inicial y finales	Cuadro combinado para el inicial y lista de casillas para los finales
R6	Dibujar estados y transiciones	Control PanelAutomata con GDI+ (círculos, flechas, bucles, etiquetas)
R7	Identificar inicial y finales	Flecha «inicio» hacia el estado inicial y doble círculo en los finales

N.º	Requisito del enunciado	Cómo se cumple
R8	Visualizar el análisis de hilos de la cadena	Lista de hilos, tabla de conjuntos activos por paso, resaltado en el dibujo y botones de paso (anterior y siguiente)
R9	Datos del estudiante al iniciar (video)	Cabecera del formulario con proyecto, estudiante, carné y fecha de creación

10.3 Diseño

Estructura de datos

El modelo es la clase `Core.Automata`, independiente de la interfaz. La tabla de transiciones se guarda como un diccionario anidado: estado → símbolo → lista de estados destino. Esa misma estructura sirve para el AFD (listas de un elemento) y para el AFND (listas de varios elementos, y el símbolo especial ε para transiciones vacías).

Miembro	Descripción
EsAFND, UsaEpsilon	Tipo de autómata y si la tabla tiene columna ε
Estados, Alfabeto	Listas de nombres (q0, q1, ...) y de símbolos de un carácter
Inicial, Finales	Estado inicial y conjunto de estados finales
FijarTransicion, Destinos	Escritura y lectura de una celda (estado, símbolo)
Aristas()	Agrupa las transiciones por par (origen, destino); un bucle con a y b se dibuja una sola vez con la etiqueta «a,b»
Validar()	Devuelve los problemas de la definición: sin estados o alfabeto, sin inicial, destinos inexistentes y, en un AFD, celdas con más de un destino
CierreEpsilon(conjunto)	Estados alcanzables solo con transiciones ε
Analizar(cadena)	Devuelve un ResultadoCadena: aceptación, conjuntos de estados por paso y la lista de hilos

Análisis de la cadena e hilos

Un hilo es un camino posible dentro del autómata para la cadena. En un AFD siempre hay uno solo; en un AFND cada transición con varios destinos (o con ε) ramifica el recorrido. `Analizar` hace dos cosas:

- **Simulación por subconjuntos:** parte de la clausura-ε del estado inicial y, por cada símbolo, calcula el conjunto de estados alcanzados más su clausura-ε. La cadena se acepta si el conjunto final contiene algún estado final. Es la tabla «Estados activos» de la pantalla.
- **Enumeración de hilos:** una búsqueda en profundidad recorre todos los caminos. Cada hilo termina como Aceptado (consumió toda la cadena y está en un estado final), Rechazado (consumió toda la cadena en un estado no final) o Bloqueado (no existe transición para el siguiente símbolo). Para evitar ciclos infinitos de transiciones ε se recuerdan los estados ya visitados sin consumir entrada, y la lista se limita a 300 hilos.

Los símbolos de la cadena que no pertenecen al alfabeto se reportan como error antes de simular.

Algoritmo de dibujo

- Los estados se colocan sobre una elipse ajustada al tamaño del panel; el estado inicial queda siempre a la izquierda.
- Cada arista es una curva de Bézier que empieza y termina en el borde de los círculos. Si existe también la transición inversa, ambas se curvan hacia lados opuestos para no encimarse.
- Un bucle se dibuja hacia afuera del círculo; las transiciones solo con ε se dibujan punteadas.
- El estado inicial lleva una flecha con la palabra «inicio» y los finales un segundo círculo interior.
- Al seleccionar un hilo se resaltan en naranja las aristas recorridas con el número de paso, y el estado actual se marca con un contorno grueso; los estados activos de un paso se rellenan de verde.

Fragmentos clave del código

```
// Core/Automata.cs - enumeración de hilos (búsqueda en profundidad)
private void Explorar(string estado, int pos, string cadena, Hilo camino,
                      HashSet<string> epsVisitados, ResultadoCadena r)
{
    if (r.Hilos.Count >= MaximoHilos) { r.Truncado = true; return; }
    bool alFinal = pos == cadena.Length;
    if (alFinal && Finales.Contains(estado))
    { Cerrar(camino, r, true, "Aceptado", pos); return; }
    bool hayMovimiento = false;
    if (!alFinal)                       // transiciones que consumen un símbolo
        foreach (string d in Destinos(estado, cadena[pos].ToString()))
            { /* agrega el paso al camino y llama a Explorar */ }
    if (UsaEpsilon && !epsVisitados.Contains(estado))   // transiciones ε
        foreach (string d in Destinos(estado, Epsilon))
            { /* agrega el paso ε y llama a Explorar sin avanzar */ }
    if (!hayMovimiento)
        Cerrar(camino, r, false, alFinal ? "Rechazado" : "Bloqueado", pos);
}
```

```
// Views/PanelAutomata.cs - arista curva entre dos estados
PointF medio = new PointF(
    (c1.X + c2.X) / 2 + perp.X * desplazamiento,   // punto de control
    (c1.Y + c2.Y) / 2 + perp.Y * desplazamiento);
PointF ini = Sumar(c1, Normalizar(medio - c1), r); // borde del círculo origen
PointF fin = Sumar(c2, Normalizar(medio - c2), r); // borde del círculo destino
g.DrawBezier(pen, ini, cp1, cp2, fin);
Flecha(g, medio, fin, color, ancho);               // punta según la tangente
```

10.4 Adaptabilidad según AFD o AFND

El formulario cambia de comportamiento según la opción elegida:

Aspecto	AFD	AFND
Contenido de cada celda	Un solo estado (ej. q1)	Uno o varios estados separados por coma (ej. q0,q1)

Aspecto	AFD	AFND
Columna ε	No disponible (casilla desactivada)	Opcional con la casilla «Incluir transiciones ε»
Texto de ayuda y título	Explica que una celda vacía bloquea la cadena	Explica la lista de estados y la columna ε
Validación	Rechaza celdas con más de un destino	Acepta varios destinos
Ejemplo que carga el botón	AFD que acepta cadenas que terminan en «ab»	AFND «termina en ab» y, con ε, el AFND de Σ*ab
Hilos al analizar	Un único hilo	Varios hilos; cada uno marcado como aceptado, rechazado o bloqueado
Dibujo	Rotulado «AFD»	Rotulado «AFND» (y «con transiciones ε»); aristas ε punteadas

Al cambiar el tipo, n, m, el alfabeto o la casilla ε, la tabla se reconstruye conservando lo ya escrito en las celdas que siguen existiendo. También se puede escribir «1» o «Q1» para referirse a q1.

10.5 Manual de uso del módulo

1. En la ventana principal pulse **Autómatas AFD / AFND**.
2. Elija **AFD** o **AFND** (y, en AFND, marque ε si desea transiciones vacías).
3. Indique **n** (número de estados) y **m** (número de símbolos) y, si lo desea, escriba el alfabeto, por ejemplo `a,b`.
4. Llene la **tabla de transiciones**: cada renglón es un estado y cada columna un símbolo.
5. Seleccione el **estado inicial** y marque los **estados finales**.
6. El autómata se dibuja automáticamente; también puede pulsar **Dibujar autómata**. Si la definición tiene problemas, un aviso bajo la tabla los describe.
7. Escriba una **cadena de entrada** y pulse **Analizar cadena**: se muestra si es aceptada o rechazada, la lista de hilos, y los conjuntos de estados por paso.
8. Seleccione un hilo y use los botones de paso (anterior y siguiente) para recorrerlo paso a paso en el dibujo. Con **Guardar PNG** se exporta el dibujo.

El botón **Cargar ejemplo** llena todo con un autómata de muestra para probar rápidamente.

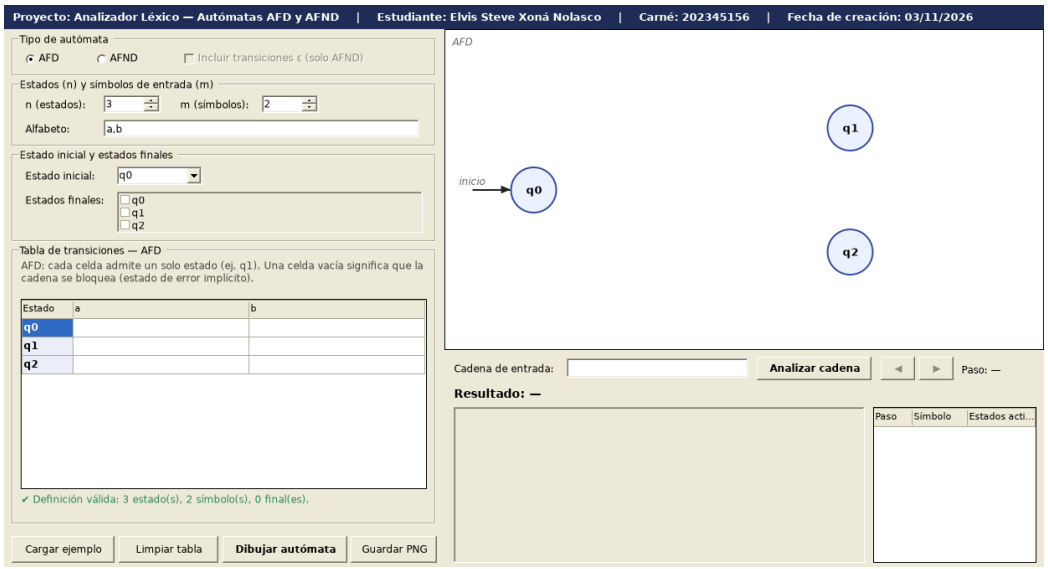


Figura 19. Formulario al abrirse: encabezado con los datos del proyecto, tipo AFD y tabla de 3 estados con alfabeto a,b.

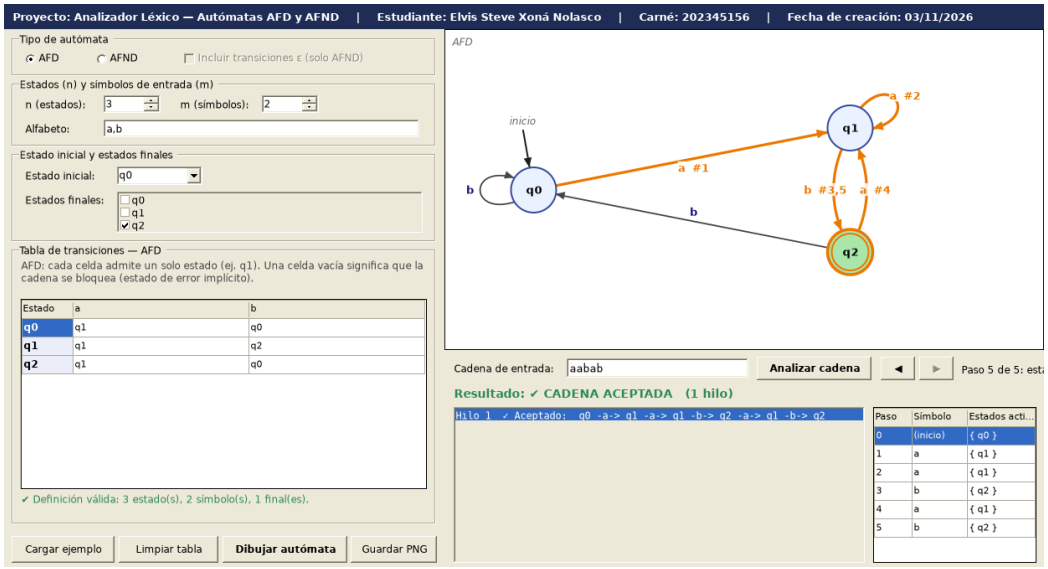


Figura 20. AFD «termina en ab»: la cadena aabab es aceptada; se resalta el recorrido con el número de paso de cada transición.

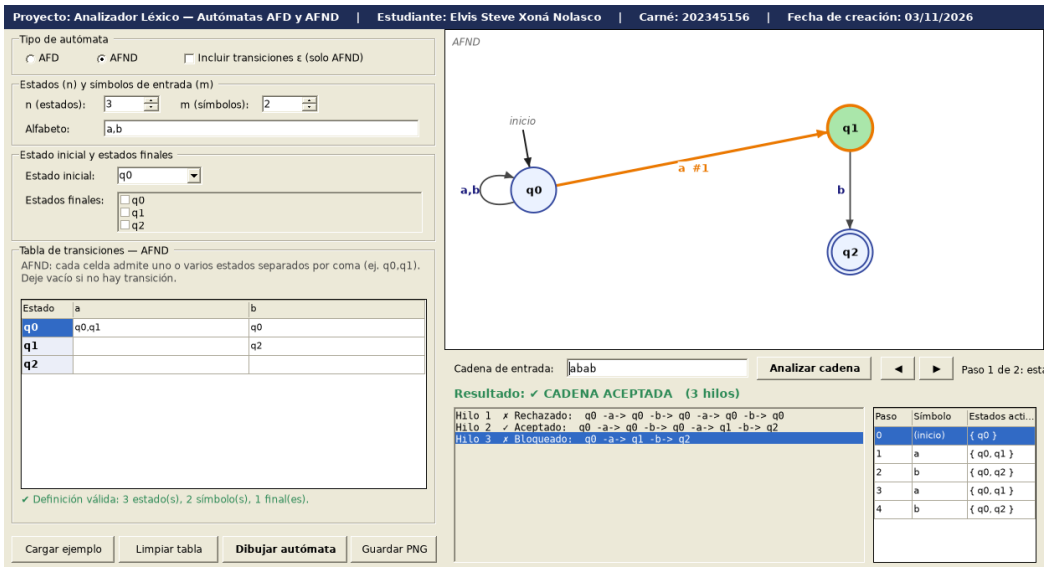


Figura 21. AFND «termina en ab» con la cadena abab: tres hilos (uno rechazado, uno aceptado y uno bloqueado). Se muestra el paso 1 del tercer hilo.

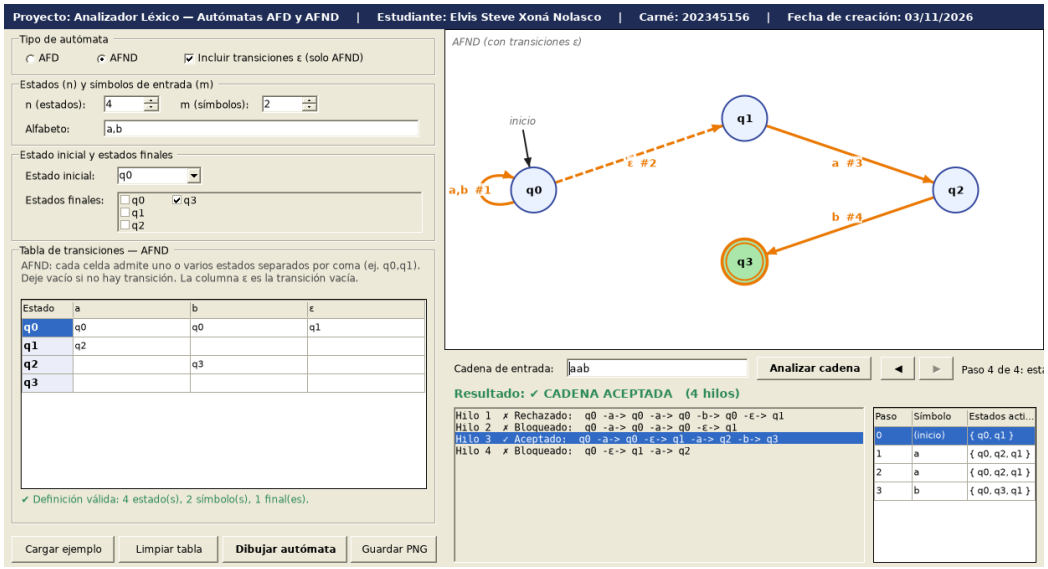


*Figura 22. AFND con transiciones ε (Σ*ab): la transición ε se dibuja punteada y el hilo aceptado usa q0 → ε → q1 → a → q2 → b → q3.*

10.6 Ejemplos de análisis

Autómata	Cadena	Hilos	Resultado
AFD «termina en ab»	aab	1	Aceptada: q0 -a-> q1 -a-> q1 -b-> q2
AFD «termina en ab»	abb	1	Rechazada: q0 -a-> q1 -b-> q2 -b-> q0
AFD «termina en ab»	(vacía)	1	Rechazada: q0 no es final
AFD «termina en ab»	abc	0	Error: el símbolo 'c' no pertenece al alfabeto {a, b}
AFND «termina en ab»	ab	2	Aceptada; un hilo rechazado (q0 -a-> q0 -b-> q0) y uno aceptado (q0 -a-> q1 -b-> q2)

Autómata	Cadena	Hilos	Resultado
AFND «termina en ab»	aab	3	Aceptada; un hilo se bloquea en q1 (sin transición con a)
AFND «termina en ab»	ba	2	Rechazada; ningún hilo termina en q2
AFND con ε (Σ*ab)	aab	4	Aceptada; el hilo aceptado usa la transición ε de q0 a q1

10.7 Cumplimiento de los criterios de calificación

Criterio	Sí	Evidencia
1. Funcionalidad (40 %): correcta implementación del formulario	Sí	Sección 10.5, figuras 19 a 22
Correcta generación de la tabla de transiciones con los datos ingresados	Sí	DataGridView generado según n, m, alfabeto y ε; pruebas T26 y T27
Precisa visualización gráfica de la tabla	Sí	PanelAutomata; figuras 20 a 22
2. Interfaz de usuario (20 %): facilidad de uso	Sí	Redibujo automático, avisos de validación, ejemplos y PNG
Diseño limpio y claro del menú y del formulario	Sí	Botón en el menú principal y formulario con grupos rotulados
3. Adaptabilidad (20 %): formulario según AFD o AFND	Sí	Sección 10.4
4. Documentación (10 %): explicación del código y comentarios	Sí	Sección 10.3, comentarios XML en el código y secciones 11 y 12
5. Video (10 %): datos del estudiante al iniciar	Sí	Cabecera del formulario con proyecto, estudiante, carné y fecha

11. Manual técnico

11.1 Requisitos y compilación

- Windows, Visual Studio 2019/2022 con la carga de trabajo de escritorio .NET y .NET Framework 4.7.2.
- Abrir Analizador Lexico.sln; proyecto de inicio Analizador Lexico (F5) para la aplicación o Analizador Lexico.Pruebas (Ctrl+F5) para las pruebas.
- Los archivos nuevos (Core\Automata.cs, Core\VerificadorEstructura.cs, Views\FrmDiccionario, Views\DlgExportar, Views\FrmAutomatas, Views\PanelAutomata.cs y Views\DatosProyecto.cs) ya están incluidos en el .csproj. Los formularios tienen su archivo .Designer.cs y se abren en el diseñador.

11.2 Flujo de datos

Texto del editor → `AnalizadorLexico.Analizar()` → `VerificadorEstructura.Verificar()` → `ResultadoAnalisis` (tokens, símbolos, errores, estadísticas) → `FrmPrincipal` llena las tablas → `ResaltadorSintaxis` colorea el editor → `Exportador` genera archivos. Una edición en una tabla reemplaza texto en el editor y repite el flujo.

Módulo de autómatas: `FrmAutomatas` lee la tabla y los controles → arma un `Automata` → `Validar()` → `PanelAutomata` lo dibuja → `Automata.Analizar(cadena)` devuelve los hilos → `FrmAutomatas` los muestra y resalta el recorrido en el panel.

11.3 Archivos entregados

Archivo / carpeta	Contenido
EntregaFinal_... .pdf	Este informe
Analizador_Lexico_Final_v4.zip	Solución completa de Visual Studio (código fuente, proyecto de pruebas, archivos de prueba)
Docs\Diccionarios\ (4 archivos)	diccionario_basico_24.txt, diccionario_ampliado.txt, diccionario_minimo.csv, diccionario_pseudocodigo_es.csv
Docs\Prueba_*.txt	Archivos de prueba del analizador léxico
Analizador Lexico.Pruebas\resultados_pruebas.txt	Resultado de la suite de 44 casos

12. Manual de usuario

12.1 Instalación y ejecución

No requiere instalación: compile con Visual Studio y ejecute Analizador Lexico.exe desde bin\ Debug. Funciona sin archivos externos.

12.2 Uso paso a paso

1. Cargue código: Abrir archivo, arrastre un archivo al editor o escriba el código.
2. Pulse Analizar (F5).
3. Revise las pestañas Tokens, Errores, Tabla de símbolos y Estadísticas; un clic en una fila lleva al lugar exacto en el editor y la edición de una celda cambia el código.
4. Use Diccionario para cambiar las palabras reservadas.
5. Exporte con Exportar (Ctrl+E): todo, o solo tokens, errores, símbolos o estadísticas.
6. Pulse Autómatas AFD / AFND para abrir el módulo de tablas de transiciones (sección 10.5).

13. Conclusiones

- Un único AFD con estado inicial central resultó suficiente y eficiente para todas las categorías, y su refinamiento en la implementación evitó aceptar números mal formados.
- Separar Core de Views permitió validar el motor con pruebas de consola antes de construir la interfaz, reutilizarlo sin cambios y agregar después el verificador de estructura y el módulo de autómatas sin tocar el autómata léxico.
- La recuperación de errores por regla permite reportar varios errores en una sola pasada sin detener el análisis; las reglas 8 a 11 ahora cubren también delimitadores, puntos y coma y operadores faltantes.
- Convertir la lista de palabras reservadas en un diccionario editable muestra que el vocabulario es un dato del analizador y no parte de su código.

- Representar AFD y AFND con una misma tabla de transiciones permitió usar un solo modelo, un solo dibujo y un solo algoritmo de análisis; la diferencia entre ambos es la cantidad de destinos por celda y la enumeración de hilos.
- Una batería de pruebas con valores esperados derivados de la especificación detecta diferencias entre el diseño y la implementación; la suite creció de 16 a 44 casos.

Trabajo futuro

- Ampliar las palabras reservadas a todas las de C# (listadas en el Entregable 1) y guardar el diccionario entre sesiones.
- Literales hexadecimales, notación científica y cadenas verbatim.
- Pruebas unitarias con MSTest/NUnit además de la suite de consola.
- Convertir un AFND a AFD (construcción de subconjuntos), minimizar un AFD y permitir símbolos de más de un carácter.
- Análisis sintáctico como siguiente fase del compilador.

14. Referencias bibliográficas

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). Compiladores: Principios, técnicas y herramientas (2a ed.). Pearson Educación.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introducción a la teoría de autómatas, lenguajes y computación (3a ed.). Addison-Wesley.
- Microsoft Corporation. (2023). Especificación del lenguaje C# y documentación de .NET. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/csharp/
- Sipser, M. (2013). Introduction to the Theory of Computation (3rd ed.). Cengage Learning.

15. Repositorio

- https://github.com/Gabo17j/Analizador-Lexico
- https://youtu.be/DDzgi0eGfE0